

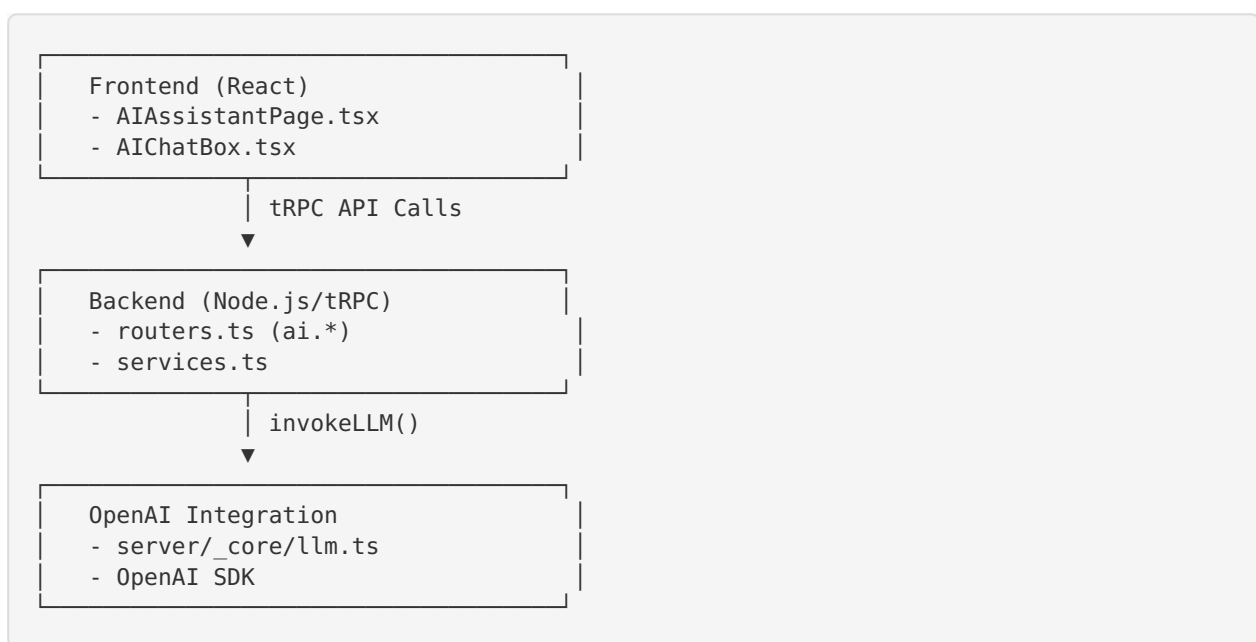
OpenAI Integration Documentation

Overview

Das Portfolio-Dashboard nutzt OpenAI's GPT-Modelle für KI-gestützte Portfolio-Analysen, Aktienempfehlungen und Sparplan-Optimierung. Diese Dokumentation beschreibt die technische Implementierung und Best Practices.

Architektur

Layer-Übersicht



Komponenten

1. OpenAI Client (server/_core/llm.ts)

Der zentrale OpenAI-Client mit robusten Error Handling und Model-Fallback.

Features

- **Multi-Model Support:** Automatischer Fallback zwischen GPT-4o → GPT-4o Mini → GPT-4 Turbo → GPT-3.5 Turbo
- **Robustes Error Handling:** Detaillierte Fehlerbehandlung mit spezifischen Error-Codes
- **Type Safety:** Vollständig typisierte Message-Struktur
- **Validation:** Triple-Layer Validation für API-Anfragen
- **Logging:** Umfassendes Debug-Logging für Troubleshooting

Message Format

```
export type Message = {
  role: "system" | "user" | "assistant" | "tool" | "function";
  content: string | TextContent | TextContent[];
  name?: string;
};
```

Usage Example

```
import { invokeLLM } from "../server/_core/llm";

const messages = [
  {
    role: "system",
    content: "Du bist ein erfahrener Finanzberater."
  },
  {
    role: "user",
    content: "Analysiere mein Portfolio"
  }
];

const response = await invokeLLM(messages);
console.log(response);
```

2. Backend Services (server/services.ts)

Business Logic für Portfolio-Analysen mit OpenAI.

Funktionen

analyzePortfolio()

Vollständige Portfolio-Analyse mit KI.

```
export async function analyzePortfolio(
  userId: number,
  positions: any[],
  customQuestion?: string,
  watchlist?: any[]
): Promise<{ analysis: string; type: string }>
```

Features:

- Berechnet Gesamtwert, Gewinn/Verlust, Allokation
- Identifiziert Top/Worst Performer
- Berücksichtigt Watchlist für Empfehlungen
- Strukturierte Prompt-Engineering für deutsche Antworten

Prompt-Struktur:

System: Rollen-Definition + Formatierungs-Vorgaben
User: Portfolio-Daten + Spezifische Frage

generateRecommendation()

Aktienspezifische Empfehlungen.

```
export async function generateRecommendation(
  userId: number,
  ticker: string,
  name: string,
  currentPositions: any[]
): Promise<{ recommendation: string; action: string }>
```

Features:

- Unternehmensanalyse
- Chancen/Risiken
- Konkrete Kauf/Halten/Verkauf-Empfehlung
- Portfolio-Allokations-Vorschläge

3. tRPC API Routen (server/routers.ts)

API-Endpunkte für Frontend-Zugriff.

Verfügbare Routen

```
ai: router({
  // Portfolio-Analyse
  analyzePortfolio: protectedProcedure.mutation(),

  // Aktien-Empfehlung
  getRecommendation: protectedProcedure
    .input(z.object({ ticker: z.string(), name: z.string() }))
    .mutation(),

  // Freier Chat
  chat: protectedProcedure
    .input(z.object({ message: z.string() }))
    .mutation(),

  // Sparplan-Empfehlung
  suggestSparplan: protectedProcedure
    .input(z.object({
      monthlyBudget: z.number(),
      currentAllocations: z.array(...)
    }))
    .mutation(),
})
```

4. Frontend-Komponenten

AIChatBox.tsx

Wiederverwendbare Chat-Komponente mit Streamdown-Markdown-Rendering.

Features:

- Streaming-Antworten
- Markdown-Rendering
- Auto-Scroll
- Loading States
- Suggested Prompts

Usage:

```
import { AIChatBox } from "@components/AIChatBox";

<AIChatBox
  messages={messages}
  onSendMessage={handleSend}
  isLoading={chatMutation.isPending}
  suggestedPrompts={["Analysiere mein Portfolio", "Risiken"]}
/>
```

AIAssistantPage.tsx

Vollständige KI-Assistent Seite mit Quick Actions.

Features:

- Quick Action Buttons (Analysieren, Risiken, Diversifikation, Empfehlungen)
- Chat-Interface
- Info-Card mit Disclaimer

Environment Setup

Erforderliche Environment Variable

```
OPENAI_API_KEY=sk-proj-...
```

Wie man einen API Key erhält:

1. Registrierung bei [OpenAI Platform](https://platform.openai.com/) (https://platform.openai.com/)
2. Navigiere zu API Keys
3. Erstelle neuen Secret Key
4. Füge Key zu `.env` hinzu

Railway Deployment

1. Navigiere zu Railway Dashboard → Projekt
2. Settings → Variables
3. Füge `OPENAI_API_KEY` hinzu
4. Deployment wird automatisch neu gestartet

Error Handling

Error Types

1. API Key Missing

```
Error: "OpenAI API key is missing. AI features are disabled."
```

Lösung: `OPENAI_API_KEY` in Environment Variables setzen

2. Model Not Found

```
Error: "OpenAI API Error: model does not exist (Model: gpt-4o, Code: model_not_found)"
```

Automatischer Fallback: System versucht automatisch nächstes Modell

3. Rate Limit

Error: "OpenAI API Error: rate_limit_exceeded (Code: rate_limit_exceeded)"

Lösung: Warten oder Upgrade auf höheres OpenAI-Tier

4. Quota Exceeded

Error: "OpenAI API Error: quota exceeded (Code: insufficient_quota)"

Lösung: OpenAI-Konto aufladen oder Billing aktivieren

User-Friendly Error Messages

Alle Errors werden in benutzerfreundliche deutsche Nachrichten übersetzt:

```
const userMessage = errorMessage.includes("API key")
  ? "OpenAI API-Schlüssel fehlt oder ist ungültig."
  : errorMessage.includes("quota") || errorMessage.includes("rate_limit")
  ? "OpenAI API-Limit erreicht. Bitte versuchen Sie es später erneut."
  : `Die KI-Analyse ist derzeit nicht verfügbar. Fehler: ${errorMessage}`;
```

Best Practices

1. Prompt Engineering

System Prompts

Definiere klar die Rolle und Erwartungen:

```
const systemPrompt = `Du bist ein erfahrener Finanzberater und Portfolio-Analyst.
Du analysierst Portfolios und gibst fundierte, aber verständliche Empfehlungen auf
Deutsch.
Sei konkret und gib praktische Handlungsempfehlungen.
Formatiere deine Antwort übersichtlich mit Überschriften und Tabellen.`;
```

User Prompts

Strukturiere Informationen klar:

```
const userPrompt = `
Portfolio-Übersicht:
- Gesamtwert: ${totalValue.toFixed(2)} €
- Gewinn/Verlust: ${totalGainPercent.toFixed(2)}%

Allokation nach Typ:
${Object.entries(byType).map(...).join('\n')}

Frage: ${customQuestion}
`;
```

2. Token Management

- Nutze `max_tokens: 2000` für detaillierte Analysen
- Kürzere Antworten für einfache Fragen

- Reduziere Portfolio-Daten auf Wesentliches

3. Model Selection

```
const MODELS_TO_TRY = [
  "gpt-4o",           // Beste Qualität, teurer
  "gpt-4o-mini",      // Gutes Preis-Leistungs-Verhältnis
  "gpt-4-turbo",      // Schnell, gute Qualität
  "gpt-3.5-turbo",    // Günstig, Fallback
];
```

Empfehlung: Starte mit GPT-4o für beste Ergebnisse

4. Logging

Aktiviere detailliertes Logging für Debugging:

```
console.log(`Attempting to use OpenAI model: ${model}`);
console.log(`Messages being sent:`, JSON.stringify(messages, null, 2));
```

In Production: Logs über Railway Dashboard einsehbar

Testing

Lokales Testing

```
# Terminal 1: Backend starten
pnpm dev

# Terminal 2: Frontend testen
curl http://localhost:3000/api/ai/analyzePortfolio
```

Integration Testing

```
// test/ai-integration.test.ts
import { analyzePortfolio } from "../server/services";

describe("AI Integration", () => {
  it("should analyze portfolio successfully", async () => {
    const result = await analyzePortfolio(1, mockPositions);
    expect(result.analysis).toBeDefined();
    expect(result.type).toBe("portfolio");
  });
});
```

Monitoring & Debugging

Railway Logs

```
# Im Browser
https://railway.app/project/<PROJECT_ID>/deployments

# CLI
railway logs
```

Was zu suchen:

-  "Successfully generated response using model: gpt-4o"
-  "Error with model gpt-4o: ..."
-  "Trying next model..."

Common Issues

Problem	Symptom	Lösung
Keine API Key	"API key is missing"	Environment Variable setzen
Invalid API Key	401 Unauthorized	Key validieren/neu generieren
Rate Limit	429 Error	Warten oder Tier upgraden
Model Error	"does not exist"	Automatischer Fallback aktiv

Kosten-Optimierung

Pricing (Stand Januar 2026)

- **GPT-4o**: ~\$0.01 per 1K tokens
- **GPT-4o Mini**: ~\$0.003 per 1K tokens
- **GPT-3.5 Turbo**: ~\$0.0005 per 1K tokens

Tipps

1. **Caching**: Speichere Analysen in DB (bereits implementiert via `saveAiAnalysis()`)
2. **Batching**: Fasse ähnliche Anfragen zusammen
3. **Model-Wahl**: Nutze GPT-4o Mini für einfache Aufgaben
4. **Token-Limit**: Setze `max_tokens` angemessen

Security

API Key Protection

DO:

- Speichere Key nur in Environment Variables
- Nutze Railway's Secret Variables
- Rotiere Keys regelmäßig

❌ DON'T:

- Niemals Key in Code committen
- Niemals Key in Frontend exponieren
- Niemals Key in Logs ausgeben

Rate Limiting

Implementiere serverseitiges Rate Limiting:

```
// Beispiel: Max 10 AI-Anfragen pro User pro Stunde
const userRequestCount = await redis.get(`ai_requests:${userId}`);
if (userRequestCount > 10) {
  throw new Error("Rate limit exceeded");
}
```

Roadmap**Geplante Features**

- [] **Streaming Responses:** Real-time Antworten mit SSE
- [] **Context Memory:** Konversations-Historie über mehrere Nachrichten
- [] **Fine-Tuning:** Custom Model für Portfolio-Analysen
- [] **Multi-Language:** Unterstützung für Englisch und weitere Sprachen
- [] **Voice Input:** Spracheingabe über Web Speech API

Verbesserungen

- [] Response Caching mit Redis
- [] A/B Testing verschiedener Prompts
- [] User Feedback Collection
- [] Automated Testing Suite
- [] Performance Monitoring mit Prometheus

Support

Bei Fragen oder Problemen:

1. Prüfe Railway Logs
2. Validiere Environment Variables
3. Teste mit `curl` direkt
4. Erstelle Issue im Repository

Letzte Aktualisierung: Januar 2026

Version: 1.0.0

Maintainer: Portfolio Dashboard Team